

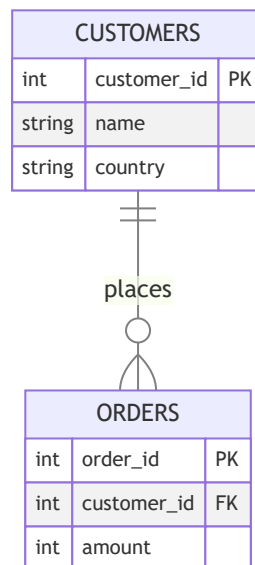
07 · Joins and Set Operators

Why this matters in interviews. Joins are the single most-tested SQL topic. Expect to draw the Venn diagrams, write the query for "customers with no orders", explain "ON vs WHERE" for outer joins, and say what a **self join** and a **cross join** are for.

In this chapter: why joins exist · INNER, LEFT, RIGHT, FULL · ON vs WHERE · CROSS JOIN · SELF JOIN · finding rows with **no match** · joining three tables · Oracle's old (+) syntax · UNION / UNION ALL and friends.

7.1 Why joins exist

Well-designed databases keep each fact **once**, in its own table, and link tables by keys: an order stores a `customer_id`, not the customer's name and address. A **join combines rows from two or more tables based on a related column between them** — typically a **foreign key in one table that references a primary key in another**.



Our two example tables (from the course):

customers

customer_id	name	country
1	Alice	USA
2	Bob	UK
3	Charlie	India

orders

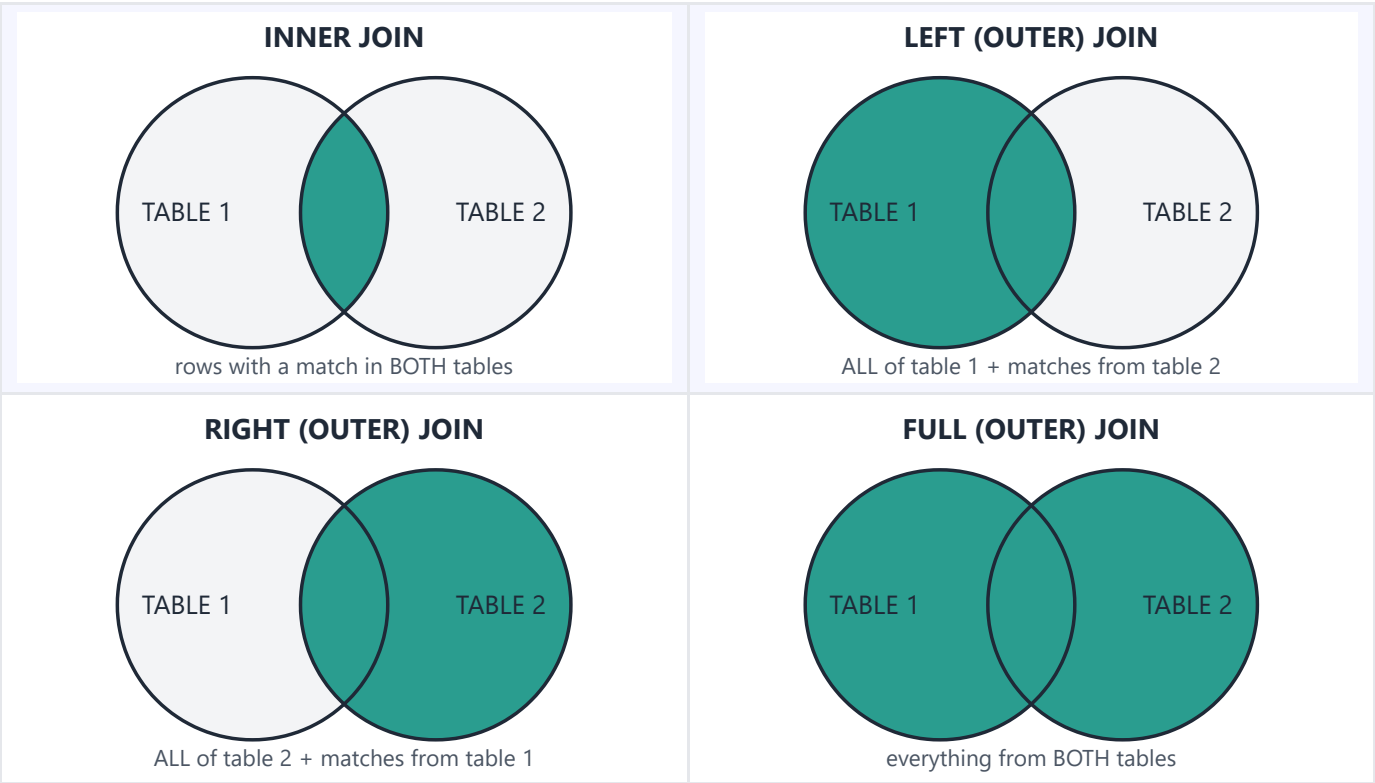
order_id	customer_id	amount
101	1	300
102	2	150

order_id	customer_id	amount
103	1	200

Alice (1) has two orders, Bob (2) has one, and **Charlie (3) has none** — exactly what makes the join types differ.

7.2 The join family at a glance

Join	Returns
(INNER) JOIN	records that have matching values in both tables
LEFT (OUTER) JOIN	all records from the left table, plus matches from the right (NULLs where no match)
RIGHT (OUTER) JOIN	all records from the right table, plus matches from the left
FULL (OUTER) JOIN	all records when there is a match in either table
CROSS JOIN	the Cartesian product — every row of the first table paired with every row of the second
SELF JOIN	a table joined with itself to compare rows within it



The words `INNER` and `OUTER` are **optional**: `JOIN` alone means `INNER JOIN` in MySQL, PostgreSQL, SQL Server and Oracle; `LEFT JOIN` = `LEFT OUTER JOIN`.

7.3 INNER JOIN

Returns rows with **matching `customer_id` in both tables**. A customer with no orders (and an order with no customer) is dropped.

```
SELECT columns
FROM   table1
INNER JOIN table2 ON table1.column = table2.column;
```

```
SELECT customers.name, orders.amount
FROM   customers
INNER JOIN orders ON customers.customer_id = orders.customer_id
ORDER BY orders.order_id;
```

Result

name	amount
Alice	300
Bob	150
Alice	200

Charlie is missing — he has no matching order.

7.4 LEFT JOIN (LEFT OUTER JOIN)

Returns **all customers, even those with no orders**. Where there is no matching order, the right-hand columns are filled with **NULL**.

```
SELECT customers.name, orders.amount
FROM   customers
LEFT JOIN orders ON customers.customer_id = orders.customer_id
ORDER BY customers.name, orders.order_id;
```

Result

name	amount
Alice	300
Alice	200
Bob	150
Charlie	NULL

Charlie appears with **NULL** as his amount. The "left" table is the one written **before** the **JOIN** keyword.

7.5 RIGHT JOIN (RIGHT OUTER JOIN)

The mirror image: returns **all rows from the right table**, matched to the left where possible. With customers/orders every order has a customer, so the result equals the inner join:

```
SELECT customers.name, orders.amount
FROM   customers
RIGHT JOIN orders ON customers.customer_id = orders.customer_id
ORDER BY orders.order_id;
```

Result

name	amount
Alice	300
Bob	150
Alice	200

(If an order had no matching customer, that row would show a `NULL` in `name`.) A `RIGHT JOIN` can always be rewritten as a `LEFT JOIN` by swapping the tables — most people only ever write `LEFT JOIN`.

7.6 FULL JOIN (FULL OUTER JOIN)

Returns **everything from both tables**, with `NULL`s where there is no match on either side. Our company data has the perfect case: **Jai has no department** and the **HR department has no employees**.

```
SELECT e.emp_name, d.dept_name
FROM   emp e
FULL JOIN dept d ON e.dept_id = d.dept_id
ORDER BY d.dept_name, e.emp_name;
```

Result

emp_name	dept_name
Bala	Engineering
Chitra	Engineering
Dinesh	Engineering
Esha	Engineering
Isha	Engineering
NULL	HR
Asha	Management
Farhan	Sales
Gita	Sales
Hari	Sales
Jai	NULL

Both *(Jai, NULL)* and *(NULL, HR)* survive — a `LEFT` join would keep only Jai's row, a `RIGHT` join only HR's.

MySQL has no `FULL JOIN`; emulate it with `LEFT JOIN ... UNION ... RIGHT JOIN`.

7.7 ON VS WHERE

- **ON** defines the **join condition** — *how* two tables are matched. It is evaluated **as the rows are combined**.
- **WHERE** **filters the result after the join**.

For **INNER JOIN** the two are interchangeable, and the **old comma syntax** puts the join condition in **WHERE** :

```
-- modern (preferred): join logic in ON, filters in WHERE
SELECT e.emp_name, d.dept_name
FROM   emp e
INNER JOIN dept d ON e.dept_id = d.dept_id
WHERE  d.location = 'Mumbai';

-- old style: works, but mixes join logic with filter logic
SELECT e.emp_name, d.dept_name
FROM   emp e, dept d
WHERE  e.dept_id = d.dept_id
AND    d.location = 'Mumbai';
```

Modern SQL prefers **ON** for join conditions and **WHERE** for extra filters — clearer, and it avoids accidental Cartesian products if you forget the join condition.

The outer-join trap

With **outer** joins the difference is huge. *Goal: every department, with its developers (if any).*

Filter in ON — the filter only decides *which employees match*; every department is kept:

```
SELECT d.dept_name, e.emp_name
FROM   dept d
LEFT JOIN emp e ON e.dept_id = d.dept_id AND e.job = 'Developer'
ORDER BY d.dept_name, e.emp_name;
```

Result

dept_name	emp_name
Engineering	Chitra
Engineering	Dinesh
HR	NULL
Management	NULL
Sales	NULL

Filter in WHERE — the filter runs *after* the join and throws away every row where **e.job** is NULL, so the departments with no developers **disappear** — the **LEFT JOIN** has silently become an **INNER JOIN** :

```

SELECT d.dept_name, e.emp_name
FROM   dept d
LEFT JOIN emp e ON e.dept_id = d.dept_id
WHERE  e.job = 'Developer'
ORDER BY d.dept_name, e.emp_name;

```

Result

dept_name	emp_name
Engineering	Chitra
Engineering	Dinesh

Rule for outer joins. A condition on the **preserved (left) table** goes in `WHERE` ; a condition on the **optional (right) table** goes in `ON` — otherwise you filter out the very rows the outer join was meant to keep.

7.8 CROSS JOIN — every combination

A **Cartesian product**: every row of table 1 paired with every row of table 2. With no relationship between the tables and a wish for **all possible combinations**, it is the right tool.

```

SELECT t1.col1, t2.col2
FROM   table1 t1
CROSS JOIN table2 t2;

```

If `table1` has **4 rows** and `table2` has **3 rows**, you get **4 × 3 = 12 rows**. Example — schedule every department for every shift:

```

CREATE TABLE shifts (shift_name TEXT);
INSERT INTO shifts VALUES ('Day'), ('Night'), ('Weekend');

SELECT d.dept_name, s.shift_name
FROM   dept d
CROSS JOIN shifts s
ORDER BY d.dept_name, s.shift_name;

```

Result

dept_name	shift_name
Engineering	Day
Engineering	Night
Engineering	Weekend
HR	Day
HR	Night

dept_name	shift_name
HR	Weekend
Management	Day
Management	Night
Management	Weekend
Sales	Day
Sales	Night
Sales	Weekend

Oracle also allows the old style, with **no WHERE linking the tables**: `FROM table1 t1, table2 t2` — which is exactly why forgetting the join condition in a comma join is dangerous.

Trap — accidental Cartesian product. Two 10,000-row tables joined without a condition yield **100,000,000 rows** and a very unhappy DBA. If a query suddenly returns "too many rows", look for a missing `ON`.

Legitimate uses: calendars × products, generating test data, pairing every size with every colour.

7.9 SELF JOIN — a table joined with itself

Used to **compare rows within the same table** — most famously *employee* ↔ *manager* when both live in one table. The trick: give the table **two different aliases** so the database can tell the two "copies" apart.

Table `employees(emp_id, name, manager_id)`:

employees

emp_id	name	manager_id
1	Alice	NULL
2	Bob	1
3	Charlie	1
4	David	2

```
SELECT E1.name AS employee, E2.name AS manager
FROM   employees E1
LEFT JOIN employees E2 ON E1.manager_id = E2.emp_id
ORDER BY E1.emp_id;
```

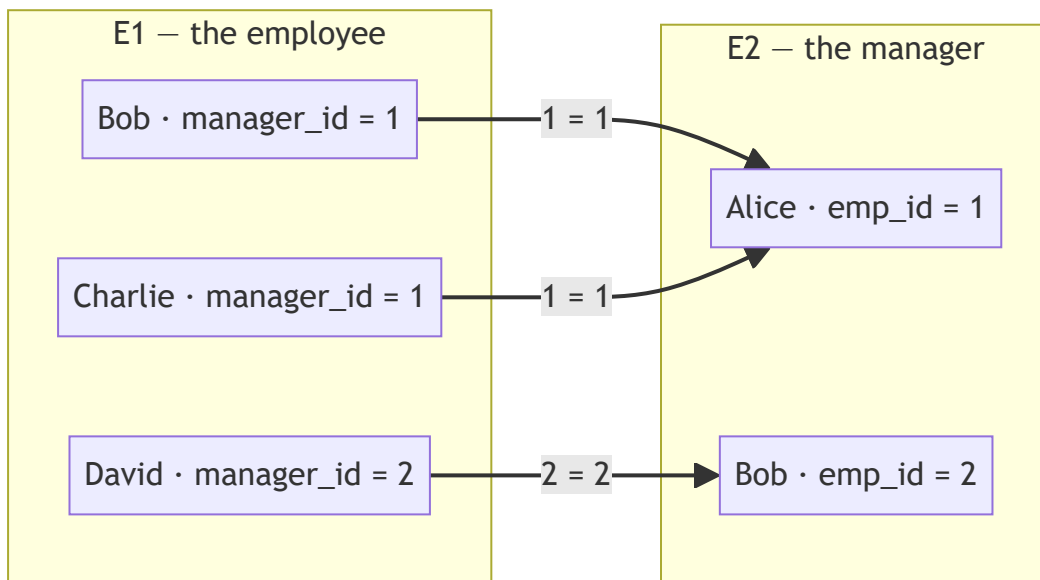
Result

employee	manager
Alice	NULL
Bob	Alice
Charlie	Alice

employee	manager
David	Bob

How to read it:

1. `FROM employees E1` — the first alias: the **employee** row (the main one).
2. `LEFT JOIN employees E2` — the **same table again**, second alias: the **manager**.
3. `ON E1.manager_id = E2.emp_id` — match each employee's `manager_id` to the manager's `emp_id`.
4. An employee with **no manager** (the CEO) gets `NULL` — that is why it is a *LEFT* join.



Use `INNER JOIN` instead if you only want employees **who have a manager**:

```
SELECT E1.name AS employee, E2.name AS manager
FROM   employees E1
INNER JOIN employees E2 ON E1.manager_id = E2.emp_id
ORDER BY E1.emp_id;
```

Result

employee	manager
Bob	Alice
Charlie	Alice
David	Bob

On the company table: each employee with their manager *and* department, three tables at once (§7.11):

```
SELECT e.emp_name AS employee, m.emp_name AS manager, d.dept_name
FROM   emp e
LEFT JOIN emp m ON e.manager_id = m.emp_id
LEFT JOIN dept d ON e.dept_id = d.dept_id
ORDER BY e.emp_id;
```

Result

employee	manager	dept_name
Asha	NULL	Management
Bala	Asha	Engineering
Chitra	Bala	Engineering
Dinesh	Bala	Engineering
Esha	Bala	Engineering
Farhan	Asha	Sales
Gita	Farhan	Sales
Hari	Farhan	Sales
Isha	Chitra	Engineering
Jai	Asha	NULL

Pairs within a group — every pair of Sales colleagues, each pair once (< avoids (A,B) and (B,A) , and avoids pairing someone with themselves):

```
SELECT a.emp_name AS colleague_1, b.emp_name AS colleague_2
FROM   emp a
JOIN   emp b ON a.dept_id = b.dept_id AND a.emp_id < b.emp_id
WHERE  a.dept_id = 30
ORDER BY a.emp_id, b.emp_id;
```

Result

colleague_1	colleague_2
Farhan	Gita
Farhan	Hari
Gita	Hari

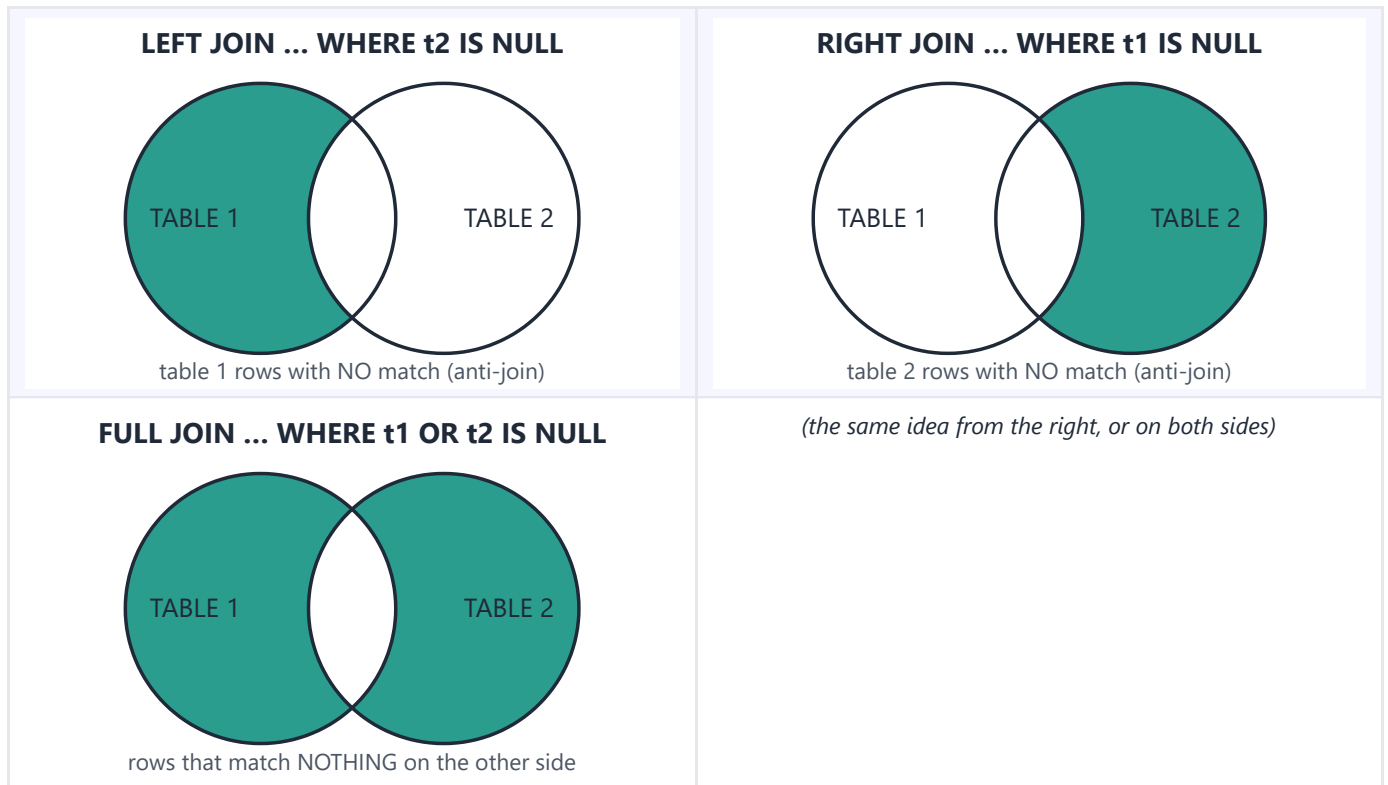
7.10 Finding rows with no match (anti-join)

A very common question: *customers with no orders, employees with no department, departments with no employees.* Take a `LEFT JOIN` and keep the rows where the **right side's key is NULL**:

```
SELECT d.dept_name
FROM   dept d
LEFT JOIN emp e ON e.dept_id = d.dept_id
WHERE  e.emp_id IS NULL;
```

Result

dept_name
HR



Test a column that **cannot be NULL in a real match** (the right table's primary key). The same result can be written with `NOT EXISTS` (usually the clearest) or `NOT IN` — see Chapter 08.

7.11 Joining more than two tables

Chain the joins; each `JOIN ... ON ...` attaches one more table to the result so far:

```
SELECT o.order_id, c.name, p.name AS product
FROM   orders o
JOIN   customers c ON o.customer_id = c.customer_id
JOIN   order_items i ON i.order_id = o.order_id
JOIN   products p  ON p.product_id = i.product_id;
```

Tips: alias every table; use `LEFT JOIN` for the *optional* links and `JOIN` for the mandatory ones; join **one relationship at a time** and check the row count after each; a chain of one-to-many joins multiplies rows, so aggregate carefully afterwards.

7.12 Oracle's legacy (+) outer join, and other syntaxes

Before Oracle 9i the outer-join operator `(+)` was the only way, and you will still meet it in old code and in Oracle interviews. Put `(+)` on the **deficient side** — the table that may have *no* matching row:

```

-- LEFT JOIN: keep all emp rows, even without a dept
SELECT e.emp_name, d.dept_name
FROM   emp e, dept d
WHERE  e.dept_id = d.dept_id (+);

-- RIGHT JOIN: keep all dept rows
WHERE  e.dept_id (+) = d.dept_id;

```

(+) cannot express a `FULL OUTER JOIN` in one query, and it is easy to get wrong when combined with other conditions — **prefer ANSI JOIN ... ON**.

Two shorthands you may see:

Syntax	Meaning
<code>JOIN t2 USING (dept_id)</code>	join on a column of the same name in both tables; that column appears once in the output
<code>NATURAL JOIN</code>	joins on all columns with the same name automatically — convenient but fragile (a new same-named column silently changes the join)

7.13 Set operators — UNION and friends

Joins combine tables **side by side** (more columns). **Set operators** stack the results of two queries **on top of each other** (more rows).

Tables:

customers

customer_name	city	country
Alfreds	Berlin	Germany
Blauer	Mannheim	Germany
Around	London	UK
Bon app	Paris	France
Ana	Berlin	Germany

suppliers

supplier_name	city	country
Exotic	London	UK
Heli	Berlin	Germany
Plutzer	Frankfurt	Germany
Tokyo Traders	Tokyo	Japan

UNION — combine, remove duplicates

```
SELECT column_name(s) FROM table1
UNION
SELECT column_name(s) FROM table2;
```

The cities (distinct) from both `customers` and `suppliers`:

```
SELECT city FROM customers
UNION
SELECT city FROM suppliers
ORDER BY city;
```

Result

city
Berlin
Frankfurt
London
Mannheim
Paris
Tokyo

If some customers and suppliers share a city, **each city is listed once**, because `UNION` selects only distinct values.

UNION ALL — combine, keep duplicates

```
SELECT city FROM customers
UNION ALL
SELECT city FROM suppliers
ORDER BY city;
```

Result

city
Berlin
Berlin
Berlin
Frankfurt
London
London
Mannheim
Paris

city
Tokyo

Nine rows: Berlin (3), London (2), Mannheim, Paris, Frankfurt, Tokyo — duplicates retained.

With WHERE

Each `SELECT` can have its own `WHERE`. *German cities only, distinct:*

```
SELECT city, country FROM customers WHERE country = 'Germany'
UNION
SELECT city, country FROM suppliers WHERE country = 'Germany'
ORDER BY city;
```

Result

city	country
Berlin	Germany
Frankfurt	Germany
Mannheim	Germany

And with `UNION ALL` the duplicates would return (Berlin ×3).

Rules

- Every `SELECT` must return the **same number of columns**, in the **same order**, with **compatible data types**.
- **Column names come from the first `SELECT`**.
- `ORDER BY` goes **once, at the very end**, and refers to the first query's column names or positions.
- **`UNION ALL` is faster** — it skips the sort/de-duplication step. Use it when you know there are no duplicates or you don't mind them.

Operator	Result	Oracle name
<code>UNION</code>	all rows from both, duplicates removed	<code>UNION</code>
<code>UNION ALL</code>	all rows from both, duplicates kept	<code>UNION ALL</code>
<code>INTERSECT</code>	rows that appear in both	<code>INTERSECT</code>
<code>EXCEPT</code>	rows in the first that are not in the second	<code>MINUS</code>

Interview corner

Q. Explain the join types with an example. Inner = only matching rows. Left = all from the left + matches (NULL where none). Right = mirror. Full = all from both. Cross = every combination. Self = a table with itself. Draw the Venn diagrams.

Q. Customers who have never placed an order? `FROM customers c LEFT JOIN orders o ON o.customer_id = c.customer_id WHERE o.order_id IS NULL (or NOT EXISTS).`

- Q. ON vs WHERE — does it matter?** Not for inner joins. For outer joins a `WHERE` on the optional table removes the NULL-extended rows and turns the join into an inner join — put that condition in `ON`.
- Q. What is a self join and why alias?** A join of a table with itself (e.g. employee → manager). Aliases are required so the two "copies" can be told apart.
- Q. What is a Cartesian product and when is it useful?** Every row × every row (`m × n` rows). Useful for generating combinations; dangerous when a join condition is forgotten.
- Q. If table A has 5 rows and table B has 3, how many rows can each join return?** Cross join: 15. Inner join: up to 15 (only matches). Left join: at least 5. Full join: at least the larger of the two.
- Q. UNION vs UNION ALL ?** `UNION` removes duplicates (extra sort); `UNION ALL` keeps them and is faster.
- Q. Join vs subquery?** Joins combine columns from several tables in one row; subqueries compute a value or set to filter/derive from. Many problems can be written both ways; joins are usually easier for the optimizer, `EXISTS` clearer for existence tests.
- Q. What does JOIN alone mean?** `INNER JOIN`.
- Q. What is (+) in Oracle?** The legacy outer-join operator, placed on the side that may lack a match.
-

Quick recap

- A join matches rows on a related column (FK → PK). `INNER` = matches only · `LEFT / RIGHT` = keep all of one side (NULLs for no match) · `FULL` = keep all · `CROSS` = every combination · `SELF` = table joined to itself with two aliases.
- `JOIN` = `INNER JOIN`. MySQL has no `FULL JOIN`.
- `ON` = how to join; `WHERE` = filter afterwards. **Outer join + WHERE on the optional table ⇒ becomes inner** — filter in `ON` instead.
- Anti-join: `LEFT JOIN ... WHERE right.pk IS NULL` (or `NOT EXISTS`).
- Forgetting the join condition ⇒ Cartesian product.
- `UNION` (distinct) / `UNION ALL` (all, faster) / `INTERSECT` / `MINUS` (`EXCEPT`): same column count and compatible types; `ORDER BY` last.